SAECOIN

SECURITY SPECIFICATION

Document Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Blockchain Type: Independent Layer-1
Consensus: Proof of Work
Document Status: Planning

---

1. PURPOSE

This document defines the security requirements for the SAECOIN blockchain ecosystem.

Security must protect:

- Blockchain integrity.
- Transaction validity.
- Wallet private keys.
- User funds.
- Mining infrastructure.
- Node communication.
- Database integrity.
- Network availability.
- Application interfaces.

Security must be considered from the beginning of development, not added after implementation.

---

2. SECURITY PRINCIPLES

SAECOIN must follow these principles:

1. Private keys must remain under wallet control.
2. Every node must independently validate blocks.
3. Every transaction must be independently verified.
4. Network data must never be trusted without validation.
5. Monetary calculations must use integer arithmetic.
6. Sensitive information must not be stored in plaintext.
7. Consensus-critical operations must be deterministic.
8. Security-sensitive code must receive additional review.
9. Failures must not silently corrupt blockchain state.
10. Mainnet must undergo independent security review before launch.

---

3. WALLET SECURITY

Wallet security is one of the highest priorities.

A wallet controls the private keys required to spend SAC.

3.1 Private Key Protection

Requirements:

- Generate private keys using a cryptographically secure random number generator.
- Never use predictable random values.
- Never transmit private keys to remote servers.
- Never include private keys in application logs.
- Never commit private keys to GitHub.
- Never expose private keys through debugging interfaces.
- Clear sensitive temporary data from memory where practical.

3.2 Seed Phrase Protection

If SAECOIN uses a recovery seed phrase:

- Generate it using a secure entropy source.
- Verify the backup process.
- Display the recovery phrase only when necessary.
- Warn users not to share it.
- Never send it to a server.
- Never store it in plaintext.

The final seed phrase standard must be selected and documented before wallet implementation.

3.3 Encrypted Wallet Storage

Wallet secrets must be encrypted at rest.

The encryption design must specify:

- Encryption algorithm.
- Key derivation function.
- Password handling.
- Salt generation.
- Nonce or initialization vector handling.
- Authentication and integrity verification.
- Encryption format version.

Use established, reviewed cryptographic libraries.

Do not invent a custom encryption algorithm.

3.4 Wallet Backup

Users must be able to:

- Back up wallet recovery information.
- Restore a wallet.
- Verify wallet recovery.
- Understand the consequences of losing recovery information.

Wallet backups should be encrypted when stored as files.

---

4. TRANSACTION SECURITY

Transactions must be protected against unauthorized spending and manipulation.

Requirements

- Verify digital signatures.
- Validate all transaction inputs.
- Confirm referenced UTXOs exist.
- Reject already-spent outputs.
- Reject duplicate inputs.
- Check input and output amounts.
- Prevent integer overflow.
- Validate spending conditions.
- Reject malformed transactions.

Double-Spend Protection

The UTXO model must prevent the same output from being spent more than once in the active chain.

Nodes must independently validate conflicting transactions.

Mempool conflict handling must not replace consensus validation.

---

5. BLOCKCHAIN SECURITY

Every full node must independently verify blockchain data.

Block Validation Security

Nodes must check:

- Block structure.
- Block hash.
- Proof of Work.
- Difficulty target.
- Previous block reference.
- Timestamp rules.
- Merkle root.
- Transaction validity.
- Coinbase reward.
- Block and transaction limits.

Invalid blocks must be rejected.

Chain Reorganization Security

The node must:

- Track cumulative chain work.
- Identify the common ancestor.
- Restore previous UTXO state.
- Apply the new valid chain.
- Preserve database consistency.

A chain with a greater height must not automatically be preferred over one with greater cumulative work.

---

6. PROOF-OF-WORK SECURITY

Proof of Work must be independently verifiable.

Requirements:

- Use a reviewed cryptographic hash function.
- Validate the difficulty target.
- Reject invalid proof submissions.
- Prevent integer overflow in difficulty calculations.
- Validate block timestamps.
- Enforce the correct mining reward.

The mining algorithm must undergo security and performance review before final selection.

ASIC resistance must not be treated as guaranteed.

---

7. P2P NETWORK SECURITY

The P2P network must protect nodes from malicious participants.

Network Protections

- Maximum peer connection limits.
- Message size limits.
- Connection timeouts.
- Invalid message rejection.
- Peer scoring.
- Misbehavior detection.
- Rate limiting where appropriate.
- Safe message parsing.
- Protocol version validation.

Attack Considerations

The implementation must consider:

- Sybil attacks.
- Eclipse attacks.
- Denial-of-service attacks.
- Malformed network messages.
- Peer impersonation.
- Spam transactions.
- Connection exhaustion.

Nodes should maintain connections to diverse peers.

No single peer should be trusted as the source of truth.

---

8. DATABASE SECURITY

The database stores critical blockchain information.

Requirements

- Validate data before database writes.
- Use atomic updates for related state changes.
- Detect corrupted records where practical.
- Protect against interrupted writes.

- Maintain consistent block and UTXO state.
- Implement safe recovery procedures.
- Support database backups.
- Keep sensitive wallet data separate from node storage.

Database corruption must not cause the node to silently accept invalid blockchain state.

Explorer indexes must be rebuildable from validated blockchain data.

---

9. API SECURITY

All application interfaces must validate incoming requests.

Requirements

- Input validation.
- Request size limits.
- Safe error handling.
- Rate limiting where appropriate.
- Authentication for privileged operations.
- Restricted access to administrative interfaces.
- Secure transport for remote connections.
- No exposure of sensitive wallet information.

Public read-only APIs should be separated from privileged node-control operations.

Wallet signing must remain local to the wallet.

---

10. ANDROID WALLET SECURITY

The Android wallet must follow secure mobile application practices.

Requirements:

- Use Android-supported secure storage mechanisms where appropriate.
- Encrypt sensitive wallet information.
- Avoid logging sensitive data.
- Protect backup files.
- Validate node responses.
- Avoid exposing private keys through intents or external interfaces.
- Do not hardcode secret credentials.
- Minimize unnecessary application permissions.

The application must warn users before performing high-risk operations.

---

11. WINDOWS WALLET SECURITY

The Windows wallet must:

- Store encrypted wallet secrets.
- Protect sensitive application files.
- Validate all node communication.
- Avoid exposing private keys through logs.
- Avoid hardcoded credentials.
- Restrict privileged operations.
- Provide secure backup and recovery.

Windows-specific secure storage mechanisms may be evaluated during implementation.

---

12. MINING SECURITY

Mining software must not compromise the user's device or funds.

Requirements:

- Verify mining job structure.
- Validate returned mining solutions.
- Restrict unsafe external inputs.
- Protect worker credentials where applicable.
- Avoid unnecessary administrator privileges.
- Clearly display mining activity.
- Monitor abnormal resource consumption.

Mining software must never request or collect wallet recovery phrases.

---

13. SERVER SECURITY

Public nodes and services must follow basic server security practices.

Requirements:

- Use supported operating systems.
- Apply security updates.
- Restrict unnecessary open ports.
- Use firewall rules.
- Protect SSH access.

- Use separate service accounts.
- Monitor resource consumption.
- Maintain secure backups.
- Protect administrative credentials.

Private server credentials must never be stored in source code.

---

14. GITHUB AND SOURCE CODE SECURITY

The source repository must be protected.

Requirements:

- Never commit private keys.
- Never commit passwords.
- Never commit API secrets.
- Review dependencies.
- Pin dependency versions where appropriate.
- Monitor dependency vulnerabilities.
- Review external code before integration.
- Require tests for security-sensitive changes.

Automated secret scanning and dependency vulnerability checks should be introduced.

---

15. CRYPTOGRAPHIC SECURITY

SAECOIN must use established cryptographic algorithms.

The final protocol must explicitly define:

- Hash function.
- Digital signature algorithm.
- Public-key format.
- Address encoding.
- Key derivation.
- Seed generation.
- Transaction signing format.
- Merkle tree construction.

Custom cryptographic algorithms must not be invented for production use.

Cryptographic implementations should use established libraries and undergo review.

---

16. SUPPLY AND REWARD SECURITY

The blockchain must prevent unauthorized coin creation.

Requirements:

- Enforce maximum supply.
- Validate coinbase transactions.
- Enforce block subsidy rules.
- Validate transaction fees.
- Prevent monetary overflow.
- Use integer arithmetic.
- Reject excessive miner rewards.

Every node must independently verify coin issuance.

No administrator or central server may arbitrarily create SAC through an ordinary network operation.

---

17. UPDATE AND UPGRADE SECURITY

Software updates must be controlled.

Requirements:

- Verify official release packages.
- Publish release checksums.
- Sign releases where practical.
- Document protocol changes.
- Test upgrades on development networks.
- Test upgrades on public testnet.
- Provide rollback procedures for application updates where safe.

Consensus upgrades require special care.

A software rollback must never silently change the node's interpretation of consensus history.

---

18. SECURITY MONITORING

The system should monitor:

- Invalid block attempts.

- Invalid transaction attempts.
- Abnormal peer behavior.
- Network connection failures.
- Database errors.
- Unexpected node restarts.
- Unusual resource consumption.
- Mining anomalies.

Logs must not expose:

- Private keys.
- Seed phrases.
- Wallet passwords.
- Authentication secrets.

---

19. SECURITY TESTING

Before testnet and mainnet, SAECOIN must undergo:

Unit Security Testing

Test individual cryptographic and validation functions.

Integration Security Testing

Test interactions between blockchain modules.

Fuzz Testing

Feed malformed and unexpected inputs into:

- Block parsers.
- Transaction parsers.
- P2P message handlers.
- Serialization functions.

Database Recovery Testing

Test interrupted writes and recovery procedures.

Wallet Security Testing

Test encryption, recovery, signing, and backup behavior.

Network Attack Simulation

Test resource exhaustion, malicious peers, and invalid messages in controlled environments.

Independent Security Review

A qualified independent reviewer should examine consensus, cryptography, wallet handling, and network security before mainnet.

---

20. INCIDENT RESPONSE

A security incident response process must be documented.

It should include:

1. Incident detection.
2. Impact assessment.
3. Containment.
4. Technical investigation.
5. Recovery.
6. User communication where necessary.
7. Post-incident review.
8. Preventive improvements.

Security incidents involving consensus or wallet funds must receive the highest priority.

---

21. RESPONSIBLE DISCLOSURE

A security reporting process should be established.

Researchers should have a way to report vulnerabilities privately before public disclosure.

The project should define:

- Security contact.
- Reporting instructions.
- Severity classification.
- Response expectations.
- Disclosure coordination.

A bug bounty program may be considered after the project has sufficient resources.

---

22. SECURITY RELEASE CHECKLIST

Before public testnet:

- [ ] Consensus validation tests pass.
- [ ] Transaction validation tests pass.
- [ ] P2P parsing tests pass.
- [ ] Wallet encryption is reviewed.
- [ ] Database recovery is tested.
- [ ] No known critical vulnerabilities remain.

Before mainnet:

- [ ] Independent consensus review completed.
- [ ] Cryptographic implementation reviewed.
- [ ] Wallet security reviewed.
- [ ] Network security tested.
- [ ] Fuzz testing completed.
- [ ] Supply calculations verified.
- [ ] Genesis configuration verified.
- [ ] Release integrity checks implemented.
- [ ] Incident response process documented.

---

23. FINAL SECURITY PRINCIPLES

1. Never trust unverified blockchain data.
2. Never expose wallet private keys.
3. Never store recovery phrases in plaintext.
4. Never use custom cryptography without expert review.
5. Never allow unauthorized coin issuance.
6. Never bypass consensus validation.
7. Never expose privileged node controls unnecessarily.
8. Never commit secrets to source control.
9. Test security-sensitive changes thoroughly.
10. Do not launch mainnet without independent security review.

---

24. FINAL NOTES

This document defines the proposed security requirements for SAECOIN.

It does not claim that the system is already secure or audited.

Security must be demonstrated through implementation, testing, independent review, and ongoing maintenance.

All unresolved cryptographic and protocol decisions must be finalized before production deployment.

END OF Security_Spec.md